

RetailIQ

Retail Data Platform & Revenue Analytics

Portfolio evidence report

An end-to-end demonstration of validated ingestion, Snowflake warehousing, dbt transformation, Airflow orchestration, and analytics delivery.

TRANSACTIONS	REVENUE	PROFIT MARGIN
100,000	\$33,129,143.92	16.84%

Verified baseline | Generated 2026-09-19

1. Executive summary

RetailIQ turns a synthetic retail transaction feed into a tested analytics layer. Python performs source validation, Snowflake stores the warehouse layers, dbt builds the dimensional model and marts, and Airflow coordinates the batch from validation through final reconciliation.

Verified business metrics

Metric	Verified value	Metric	Verified value
Transactions	100,000	Customers	2,000
Products / stores	27 / 8	Units sold	162,086
Revenue	\$33,129,143.92	Profit	\$5,579,892.30
Profit margin	16.84%	Average order value	\$331.29
Date range	2024-01-01 to 2025-12-31	Revenue growth	0.84%

Architecture

Source	Validation	Warehouse	Transformation	Delivery
Retail CSV	Python rules	Snowflake RAW	dbt STAGING -> INTERMEDIATE -> MARTS	Streamlit / Power BI / HTML

Airflow is the orchestration layer across validation, loading, dbt, reconciliation, and analytics checks.

2. Airflow orchestration evidence

DAG: retailiq_pipeline | Local UI: <http://localhost:8080>

Order	Task	Purpose	Status
1	start	Begin the scheduled batch	PASS
2	validate_source	Check headers, types, formulas, dates, and identifiers	PASS
3	ingest_raw_data	Persist accepted, rejected, and batch-summary artifacts	PASS
4	load_to_snowflake	Run the reviewed stage-to-RAW MERGE	PASS
5	run_dbt_build	Build models and execute dbt tests	PASS
6	run_reconciliation	Compare raw and fact row counts and revenue	PASS
7	validate_analytics	Check fact-table formulas and positive quantities	PASS
8	end	Mark the pipeline complete	PASS

The DAG uses retries, a single active run, and warehouse-side reconciliation before success.

Show it locally

`docker compose up -d` Open <http://localhost:8080> and select the green retailiq_pipeline graph.

3. dbt transformation and test evidence

dbt docs UI: <http://localhost:8081>

dbt area	Contents	Evidence
Sources	RAW transaction source	Source contract documented
Staging	stg_transactions	Type and freshness logic
Intermediate	int_transaction_enriched	Reusable transaction grain
Dimensions	Customer, product, store, date	Deterministic surrogate keys
Fact	fct_sales	One row per transaction line
Marts	KPI, customer, product, store performance	Analytics-ready outputs
Tests	Schema and singular SQL tests	50 tests; 61/61 checks passed

Generate and serve the documentation from the Airflow container:

```
dbt docs generate --project-dir /opt/retailiq/dbt --profiles-dir /opt/retailiq/docker/airflow
dbt docs serve --host 0.0.0.0 --port 8081
```

4. Analytics and presentation outputs

The same verified baseline is available through several presentation surfaces:

Surface	Access	What it demonstrates
Streamlit	Hosted public dashboard	Interactive KPI cards, charts, product, store, and customer views
Airflow	<code>http://localhost:8080</code>	Successful orchestration and green task graph
dbt docs	<code>http://localhost:8081</code>	Model lineage, columns, and test metadata
Jupyter	<code>notebooks/retailiq_analysis.ipynb</code>	Narrated analysis with charts
Static HTML	<code>reports/retailiq_report.html</code>	Portable report with no server dependency
Power BI	<code>powerbi/</code>	Semantic model, relationships, and DAX handoff

Hosted dashboard

`https://retailiq-retail-data-platform-bv7cmxx76gmya93dhfzscx.streamlit.app/`

5. Security and reproducibility

The repository contains source code, SQL, model contracts, tests, sanitized metrics, and repeatable commands. It intentionally excludes .env files, passwords, Streamlit secrets, Airflow's metadata database, dbt/target runtime output, and operational logs.

Item	Repository treatment
Synthetic data	Safe baseline; 100,000 rows generated with seed 42
Credentials	Environment variables or hosting secret store only
Airflow metadata	Docker volume only; never committed
dbt catalog and logs	Generated locally when needed; ignored by Git
Evidence	Sanitized metrics and documented commands committed

Repository reference

`https://github.com/HarishNamasivayamM/RetailIQ-retail-data-platform`

This report is generated from the committed project snapshot and can be rebuilt with `scripts/build_pdf_report.py`.